

Java 实验报告

软件学院 李宗杭 2311451

实验题目：表达式求值（变量，异常，日志）

一、设计思路

1、总体架构

Preprocessor 类：负责输入预处理，包括变量声明、赋值和表达式识别

Evaluate 类：核心计算功能，实现表达式解析和求值

异常体系：自定义异常类处理各种错误情况

日志系统：使用 SLF4J 记录程序运行状态和错误信息

2、异常体系设计

异常定义方式

定义了 5 种自定义异常，均继承自 RuntimeException

VariableUndefinedException：变量未定义

VariableUnassignedException：变量未赋值

ErrorExpressionException：表达式语法错误

OperatorUnsupportedException：不支持的运算符

UnknownErrorException：未知错误

异常处理机制

抛出机制：在各个处理环节中通过条件判断主动抛出相应异常

捕获机制：在 main 方法中集中捕获所有异常，并通过日志系统记录

异常传递：采用"快速失败"原则，一旦发现错误立即抛出异常终止处理

3、日志系统设计

日志设置方式

使用 SLF4J 作为日志门面，配合 Logback 实现

日志级别设置为 DEBUG，便于调试

在类中通过静态 Logger 实例获取日志记录器

日志记录方式

调试信息：使用 `log.debug()` 记录输入信息和计算结果

错误信息：使用 `log.error()` 记录各种异常情况

二、测试

序号	测试内容	输入	输出
1	基本整数运算	int a=10; int b=3; a+b=?	13
2	基本浮点数运算	float a=5.5; float b=2.0; a*b=?	11.00
3	混合类型运算	int a=10; float b=2.5; a+b=?	12.50
4	取模运算	int a=10; int b=3; a%b=?	1
5	复杂括号运算	int a=2; int b=3; (a+b)*(a-b)=?	-5
6	多步混合运算	int a=5; float b=2.5; int c=2; (a*b)+c=?	14.50
7	浮点数除法	float a=10.0; float b=4.0; a/b=?	2.50
8	整数除法（截断）	int a=10; int b=3; a/b=?	3
9	多变量复杂表达式	int x=4; int y=2; float z=1.5; (x*y)+(z*2)=?	11.00
10	嵌套括号运算	int a=1; int b=2; int c=3; (a+(b*c))/(b-a)=?	7
11	大整数运算	int a=1000000; int b=2000000; a+b=?	3000000
12	浮点数精度测试	float a=0.1; float b=0.2; a+b=?	0.30
13	多运算符优先级测试	int a=5; int b=2; int c=3; a+b*c=?	11
14	负数运算	int a=-10; int b=5; a+b=?	-5
15	复杂浮点运算	float a=3.14; float b=2.0; float c=1.5; (a*b)+c=?	7.78
16	变量未定义	int a=10; b+5=?	wrong - variable undefined
17	变量未赋值	int a; int b=5; a+b=?	wrong - variable unassigned
18	类型转换错误	int a=10.5; a+1=?	wrong - class cast failed
19	不支持的操作符	int a=10; int b=2; a^b=?	wrong - operator unsupported
20	括号不匹配	int a=10; (a+5*(2-1)=?	wrong - error expression
21	连续运算符	int a=10; int b=2; a++b=?	wrong - error expression
22	运算符开头	int a=10; +a=?	wrong - error expression
23	运算符结尾	int a=10; a+=?	wrong - error expression
24	除零错误	int a=10; int b=0; a/b=?	wrong - arithmetic exception
25	空表达式	int a=10; =?	wrong - error expression
26	表达式语法错误	1(10+1)=?	wrong - error expression
27	表达式语法错误	(1*)10=?	wrong - class cast failed
28	表达式语法错误	(*10)=?	wrong - error expression
29	表达式语法错误	(10+3)20*1=?	wrong - error expression
30	多错误混合	int a=10;b=20;a+(10=?	wrong - variable undefined

四、测试结果与结论

1、测试结果分析

以上测试为最终测试，故无错误出现，实际输出与理论输出均相符合，故用同一列数据展示。但在之前的测试中出现了“除 0 异常未捕获”，“变量数据类型未正确处理”，“空表达式未处理”等问题以及一些小问题，在最终测试之前均已修正，由最终测试结果看来程序已能正确运行。

2、收获

本次实验实现了一个可以接收变量定义，可以处理异常情况并具有日志系统的程序。程序大致可分为四部分：一些自定义异常；表达式计算功能；输入语句处理功能；主函数调用预处理计算表达式并捕获异常。其中计算功能为之前的实验所实现，此次仅根据异常体系在其中插入了一些异常抛出的代码。这次实验的主要工作还是在于自定义异常的编写，异常的捕获以及日志系统的功能。通过本次实验，我熟悉了与异常相关的各种知识，深刻体会了异常层层传递的机制，同时也深入学习了一下与 Maven 相关的一些知识，体会了配置文件与依赖等相关功能，也掌握了简单的日志使用方法。

```
1. package evaluation;
2.
3. import org.slf4j.Logger;
4. import org.slf4j.LoggerFactory;
5.
6. import java.text.DecimalFormat;
7. import java.util.HashMap;
8. import java.util.Scanner;
9. import java.util.Stack;
10.
11. public class Evaluation {
12.
13.     private static final Logger log = LoggerFactory.getLogger(Evaluation.class);
14.
15.     public static void main(String[] args) {
16.         while (true) {
17.             try {
18.                 log.debug(Preprocessor.preprocess().toString());
19.             } catch (ClassCastException e) {
20.                 log.error("wrong - class cast failed");
21.             } catch (VariableUndefinedException e) {
22.                 log.error("wrong - variable undefined");
23.             } catch (VariableUnassignedException e) {
24.                 log.error("wrong - variable unassigned");
25.             } catch (OperatorUnsupportedException e) {
26.                 log.error("wrong - operator unsupported");
27.             } catch (ErrorExpressionException e) {
28.                 log.error("wrong - error expression");
29.             } catch (ArithmeticException e) {
30.                 log.error("wrong - arithmetic exception");
31.             } catch (UnknownErrorException e) {
32.                 log.error(e.getMessage());
33.             }
34.         }
35.
36.     }
37. }
38.
39. // 表达式预处理
40. class Preprocessor {
41.
42.
43.     public static HashMap<String, String> variates = new HashMap<>(); // 变量
44.     public static HashMap<String, String> variablesType = new HashMap<>();
45.     public static String expression = "";
46.
47.     public static Object preprocess() {
48.         Scanner sc = new Scanner(System.in);
49.         while (true) {
```

```
50.
51.     // 读取下一行
52.     String input = sc.nextLine();
53.     String[] tokens = input.split(";");
54.
55.     // 处理各语句
56.     for (String token : tokens) {
57.         token = token.trim();
58.
59.         // 变量
60.         if (token.startsWith("int ") || token.startsWith("float ")) {
61.             String type = "";
62.             if (token.startsWith("int")) {
63.                 if (token.contains(".")) {
64.                     throw new ClassCastException();
65.                 }
66.                 token = token.substring(3);
67.                 type = "int";
68.             } else {
69.                 type = "float";
70.                 token = token.substring(5);
71.                 if (token.contains("=") && !token.contains(".")) {
72.                     token = token + ".0";
73.                 }
74.             }
75.             token = token.replace(" ", "");
76.
77.             // 存储变量
78.             String[] variatePair = token.split("=");
79.             variablesType.put(variatePair[0], type);
80.             if (variatePair.length == 2) {
81.                 variates.put(variatePair[0], variatePair[1]);
82.             } else {
83.                 variates.put(variatePair[0], null);
84.             }
85.
86.             continue;
87.         }
88.
89.         // 赋值
90.         if (token.contains("=") && !token.contains("?")) {
91.             token = token.replace(" ", "");
92.
93.             String[] variatePair = token.split("=");
94.             if (!variates.containsKey(variatePair[0])) {
95.                 throw new VariableUndefinedException();
96.             }
97.             if (variablesType.get(variatePair[0]).equals("float")) {
98.                 if (!variatePair[1].contains(".")) {
```

```

99.         variatePair[1] = variatePair[1] + ".0";
100.     }
101.     } else if (variablesType.get(variatePair[0]).equals("int")) {
102.         if (variatePair[1].contains(".")) {
103.             throw new ClassCastException();
104.         }
105.     }
106.     variates.put(variatePair[0], variatePair[1]);
107. }
108.
109.     // 表达式
110.     if (token.contains("=") && token.contains("?")) {
111.         token = token.replaceAll("[?=\s]", "");
112.
113.         if (!token.matches("[\\w+\\-*/%().]*")) {
114.             throw new OperatorUnsupportedException();
115.         }
116.         expression = token;
117.         break;
118.     }
119. }
120.
121.     // 处理到表达式, 跳出输入循环
122.     if (input.endsWith("?")) {
123.         break;
124.     }
125. }
126.
127. Object result = Evaluate.evaluate(expression);
128. if (result instanceof Float) {
129.     double value = (Float) result;
130.     result = new DecimalFormat("#.00").format(value);
131. }
132. return result;
133. }
134. }
135.
136. // 表达式计算
137. class Evaluate {
138.
139.     public static Object evaluate(String expression) {
140.         // 去除表达式末尾的=号 (如果有)
141.         if (expression.endsWith("=")) {
142.             expression = expression.substring(0, expression.length() - 1);
143.         }
144.
145.         // 将表达式转换为 tokens
146.         String[] tokens = tokenize(expression);
147.

```

```
148.         // 使用两个栈：一个用于操作数，一个用于运算符
149.         Stack<Object> values = new Stack<>();
150.         Stack<String> ops = new Stack<>();
151.
152.         int openBrackets = 0;
153.         String last = "";
154.
155.         for (int i = 0; i < tokens.length; i++) {
156.             String token = tokens[i];
157.
158.             // 跳过空 token
159.             if (token.isEmpty()) {
160.                 continue;
161.             }
162.
163.             // 处理变量
164.             if (isVariate(token)) {
165.                 if (!Preprocessor.variates.containsKey(token)) {
166.                     throw new VariableUndefinedException();
167.                 } else {
168.                     token = Preprocessor.variates.get(token);
169.                     if (token == null) {
170.                         throw new VariableUnassignedException();
171.                     }
172.                 }
173.             }
174.
175.             // 处理数字（包括负数）
176.             if (isNumber(token)) {
177.                 if (last.equals("")) {
178.                     throw new ErrorExpressionException();
179.                 }
180.                 if (token.startsWith(".") || token.endsWith(".")) {
181.                     throw new ErrorExpressionException();
182.                 }
183.                 last = "Number";
184.
185.                 values.push(parseNumber(token));
186.                 continue;
187.             }
188.
189.             // 处理左括号
190.             if (token.equals("(")) {
191.                 if (last.equals("")) || last.equals("Number")) {
192.                     throw new ErrorExpressionException();
193.                 }
194.                 last = "(";
195.                 openBrackets++;
196.
```

```

197.         ops.push(token);
198.         continue;
199.     }
200.
201.     // 处理右括号
202.     if (token.equals("(")) {
203.         if (last.equals("(") || last.equals("operator")) {
204.             throw new ErrorExpressionException();
205.         }
206.         if (openBrackets == 0) {
207.             throw new ErrorExpressionException();
208.         }
209.         last = "(";
210.         openBrackets--;
211.
212.         while (!ops.peek().equals("(")) {
213.             values.push(applyOp(ops.pop(), values.pop(), values.pop()));
214.         }
215.         ops.pop(); // 弹出左括号
216.         continue;
217.     }
218.
219.     // 处理负号作为一元运算符的情况
220.     if (token.equals("-"
221.         ") && (i == 0 || last.equals("(") || last.equals("operator") && (i + 1 < tokens.length) && isNumber(tokens[i + 1])))) {
222.         // 这是一个一元负号，不是减法运算符
223.         // 将 0 压入值栈，然后执行减法
224.         last = "operator";
225.         values.push(0);
226.         ops.push("-");
227.         continue;
228.     }
229.
230.     // 处理运算符
231.     if (token.length() == 1 && isOperator(token.charAt(0))) {
232.         if (last.equals("(") || last.equals("operator") || i == 0 || i == tokens.length -
233.             1) {
234.             throw new ErrorExpressionException();
235.         }
236.         while (!ops.isEmpty() && hasPrecedence(token, ops.peek())) {
237.             values.push(applyOp(ops.pop(), values.pop(), values.pop()));
238.         }
239.         ops.push(token);
240.         last = "operator";
241.         continue;
242.     }
243.
244.     throw new UnknownErrorException();

```

```
243.         }
244.
245.         // 处理剩余括号
246.         if (openBrackets != 0) {
247.             throw new ErrorExpressionException();
248.         }
249.
250.         // 处理剩余运算符
251.         while (!ops.isEmpty()) {
252.             values.push(applyOp(ops.pop(), values.pop(), values.pop()));
253.         }
254.
255.         // 结果应该在值栈的顶部
256.         if (values.empty()) {
257.             throw new ErrorExpressionException();
258.         }
259.         return values.pop();
260.     }
261.
262.     // 将表达式分解为 tokens（支持负数）
263.     private static String[] tokenize(String expression) {
264.         StringBuilder tokens = new StringBuilder();
265.
266.         for (int i = 0; i < expression.length(); i++) {
267.             char c = expression.charAt(i);
268.
269.             if (c == ' ') {
270.                 continue; // 跳过空格
271.             }
272.
273.             if (isOperator(c) || c == '(' || c == ')') {
274.                 // 运算符或括号前后添加空格
275.                 tokens.append(" ").append(c).append(" ");
276.             } else {
277.                 tokens.append(c);
278.             }
279.         }
280.
281.         // 分割并过滤空字符串
282.         String[] result = tokens.toString().split(" ");
283.         // 过滤空字符串
284.         int count = 0;
285.         for (String s : result) {
286.             if (!s.isEmpty()) count++;
287.         }
288.
289.         String[] filtered = new String[count];
290.         int index = 0;
291.         for (String s : result) {
```

```
292.         if (!s.isEmpty()) {
293.             filtered[index++] = s;
294.         }
295.     }
296.
297.     return filtered;
298. }
299.
300. // 检查字符是否是运算符
301. private static boolean isOperator(char c) {
302.     return c == '+' || c == '-' || c == '*' || c == '/' || c == '%' || c == '=';
303. }
304.
305. // 检查字符串是否是变量
306. private static boolean isVariate(String str) {
307.     return str.matches("[a-zA-Z_]\\w*");
308. }
309.
310. // 检查字符串是否是数字（支持负数）
311. private static boolean isNumber(String str) {
312.     if (str.isEmpty()) {
313.         return false;
314.     }
315.
316.     int dotCount = 0;
317.     int start = 0;
318.
319.     // 处理负号
320.     if (str.charAt(0) == '-') {
321.         if (str.length() == 1) {
322.             return false; // 只有负号不是数字
323.         }
324.         start = 1;
325.     }
326.
327.     for (int i = start; i < str.length(); i++) {
328.         char c = str.charAt(i);
329.
330.         if (c == '.') {
331.             dotCount++;
332.             if (dotCount > 1) {
333.                 return false; // 多个小数点
334.             }
335.         } else if (!Character.isDigit(c)) {
336.             return false;
337.         }
338.     }
339.
340.     return true;
```

```
341.     }
342.
343.     // 解析数字为 Integer 或 Float (支持负数)
344.     private static Object parseNumber(String str) {
345.         if (str.contains(".")) {
346.             return Float.parseFloat(str);
347.         } else {
348.             return Integer.parseInt(str);
349.         }
350.     }
351.
352.     // 检查 op2 是否比 op1 有更高的优先级
353.     private static boolean hasPrecedence(String op1, String op2) {
354.         if (op2.equals("(") || op2.equals(")")) {
355.             return false;
356.         }
357.         if ((op1.equals("*") || op1.equals("/") || op1.equals("%")) && (op2.equals("+") || op2.equals("-"))) {
358.             return false;
359.         }
360.         return true;
361.     }
362.
363.     // 应用运算符
364.     private static Object applyOp(String op, Object b, Object a) {
365.         // 处理类型提升
366.         if (a instanceof Integer && b instanceof Integer) {
367.             int aInt = (Integer) a;
368.             int bInt = (Integer) b;
369.
370.             switch (op) {
371.                 case "+":
372.                     return aInt + bInt;
373.                 case "-":
374.                     return aInt - bInt;
375.                 case "*":
376.                     return aInt * bInt;
377.                 case "/":
378.                     return aInt / bInt;
379.                 case "%":
380.                     return aInt % bInt;
381.             }
382.         } else {
383.             // 将操作数转换为 float
384.             float aFloat = a instanceof Integer ? (Integer) a : (Float) a;
385.             float bFloat = b instanceof Integer ? (Integer) b : (Float) b;
386.
387.             switch (op) {
388.                 case "+":
```

```
389.         return aFloat + bFloat;
390.     case "-":
391.         return aFloat - bFloat;
392.     case "*":
393.         return aFloat * bFloat;
394.     case "/":
395.         return aFloat / bFloat;
396.     case "%":
397.         return aFloat % bFloat;
398.     }
399. }
400.     throw new UnknownErrorException();
401. }
402. }
403.
404. // 变量未定义异常
405. class VariableUndefinedException extends RuntimeException {
406.     public VariableUndefinedException() {
407.     }
408.
409.     public VariableUndefinedException(String message) {
410.         super(message);
411.     }
412. }
413.
414. // 变量未初始化异常
415. class VariableUnassignedException extends RuntimeException {
416.     public VariableUnassignedException() {
417.     }
418.
419.     public VariableUnassignedException(String message) {
420.         super(message);
421.     }
422. }
423.
424. // 表达式错误异常
425. class ErrorExpressionException extends RuntimeException {
426.     public ErrorExpressionException() {
427.     }
428.
429.     public ErrorExpressionException(String message) {
430.         super(message);
431.     }
432. }
433. // 括号匹配
434. // 连续操作符
435. // 操作符与括号连接
436. // 操作符与首尾连接
437. // .前后无数字
```

```
438.
439.
440.  // 不支持的操作异常
441.  class OperatorUnsupportedException extends RuntimeException {
442.      public OperatorUnsupportedException() {
443.      }
444.
445.      public OperatorUnsupportedException(String message) {
446.          super(message);
447.      }
448.  }
449.
450.  // 未知错误
451.  class UnknownErrorException extends RuntimeException {
452.      public UnknownErrorException() {
453.      }
454.
455.      public UnknownErrorException(String message) {
456.          super(message);
457.      }
458.  }
```